

Warm up

```
int main() {  
    printf("Hello, World!\n");  
    return 0;  
}
```

- What does this program (intend to) do?
- What is missing from this program for it to compile successfully?
- At what stage of the build process would this error be detected?

Warm up

```
$ gcc -E test.c > test.i
```

```
$ gcc -S test.i > test.o
```

```
test.c:2:5: error: call to undeclared library function 'printf' with type
'int (const char *, ...)'; ISO C99 and later do not support implicit
function declarations [-Wimplicit-function-declaration]
```

```
    2 |     printf("Hello, World!\n");
      |     ^
```

```
test.c:2:5: note: include the header <stdio.h> or explicitly provide a
declaration for 'printf'
```

Precept 3: Writing and Building Simple C Programs

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

February 3, 2026

Overview

- 1 The build process
- 2 Running a C program
- 3 Code style
- 4 Summary

What does gcc do and how?

- gcc: GNU Compiler Collection
 - Translates source code written in C (or other languages) into machine code that can be executed by a computer
- Four main steps of the build process:
 - 1 Preprocessing
 - 2 Compiling
 - 3 Assembling
 - 4 Linking
- In COS 217, we use `gcc217`, which is a shorthand for `gcc -Wall -Wextra -Wno-unused-parameter -ansi -pedantic`
 - `-Wall` and `-Wextra`: enable most compiler warnings
 - `-Wno-unused-parameter`: suppress warnings about unused function parameters
 - `-ansi` and `-pedantic`: strictly enforce ANSI C standards

Preprocessing

```
gcc217 -E hello.c > hello.i
```

- Input: C source file(s): `hello.c`
- Performs text-based transformations on the source code before compilation
 - Removes comments (`/* ... */`)
 - Handles preprocessor directives (e.g., `#include`, `#define`, `#ifdef`)
 - Converts physical lines into logical lines (handles line continuations with `\`)
- Output: intermediate source file(s): `hello.i`
 - Without comments
 - Without preprocessor directives: contains declaration of `printf()` (and other functions), but missing *definition* of `printf()`

Compiling

```
gcc217 -S hello.i > hello.s
```

- Input: Preprocessed source file(s): `hello.i`
- Performs static type checking and generates assembly language based on the preprocessed C code
- Output: Assembly file(s): `hello.s`
 - Assembly language: lower-level (but still human-readable) representation of machine code
 - Specific to the target architecture (e.g., x86, ARM)
 - Still no definition of `printf()`!

Assembling

```
gcc217 -c hello.s > hello.o
```

- Input: Assembly file(s): `hello.s`
- Translates assembly code into machine code (binary format, not human-readable)
- Output: object file(s): `hello.o`
 - Machine code: binary instructions that can be executed by the CPU
 - But notably, still no definition of `printf()` (!)

Linking

```
gcc217 hello.o -o hello
```

- Input: Object file(s): `hello.o`
- Combines multiple object files and libraries into a single executable
 - Implicitly includes standard libraries
 - ◇ Linux archive (`libc.a`) that bundles various object files
 - ◇ (Or, shared libraries that are dynamically linked at runtime (`libc.so`))
 - Resolves references to external functions and variables (e.g., `printf()`)
- Output: Executable file: `hello` (or, whatever follows the `-o` flag, `a.out` by default)
 - Machine code, still not human-readable
 - Finally(!) includes the definition of `printf()`

Overview of the build process

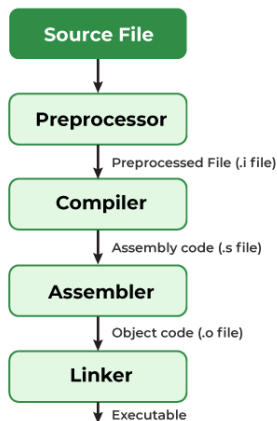


Image source:
<https://www.geeksforgeeks.org/c/compiling-a-c-program-behind-the-scenes/>

- Preprocessing
 - Removes comments and handles preprocessor directives
 - `gcc217 -E hello.c > hello.i`
- Compiling
 - Translates preprocessed C code into assembly code
 - `gcc217 -S hello.i > hello.s`
- Assembling
 - Translates assembly code into machine code (object file)
 - `gcc217 -c hello.s > hello.o`
- Linking
 - Combines object files and libraries into an executable
 - `gcc217 hello.o -o hello`
- Combined: `gcc217 hello.c -o hello`

Errors, warnings and notes

- Errors: issues that prevent successful compilation
 - e.g., syntax errors, type mismatches, undefined references
 - *Must* be fixed, otherwise executable will not be generated
- Warnings: potential issues that do not prevent compilation
 - e.g., unused variables, implicit type conversions
 - *Should* be reviewed and addressed, could point to bugs
- Notes: informational messages that provide additional context
 - May also indicate potential issues

Overview

- 1 The build process
- 2 Running a C program
- 3 Code style
- 4 Summary

Running a C program

- To run the hello program: `./hello`
 - `./` indicates that executable is located in the current directory
- When you run the program:
 - OS loads the executable into memory
 - `main` function is the entry point
 - CPU executes the machine code instructions sequentially
 - Program runs until it reaches the end of the `main` function or encounters a termination condition (e.g., an error)

What does memory look like?

- When a program runs, memory is typically divided into several segments, including:
 - Text segment: contains the compiled machine code instructions
 - ◇ Read-only, executable
 - ◇ Low addresses in memory
 - Read-only (rodata) segment: stores constant data (e.g., string literals, constants)
 - Stack: manages function calls, local variables, control flow
 - ◇ Grows and shrinks dynamically as functions are called and return
 - ◇ High addresses in memory
- Execution begins in the text segment at the `main` function
 - Can use `xxd` to inspect the binary contents (thanks to the student who suggested this in precept!)

Overview

- 1 The build process
- 2 Running a C program
- 3 Code style
- 4 Summary

Code style tips

- Avoid magic numbers (e.g., 3.14159) → use named constants
 - Macros: `#define PI 3.14159` → we will avoid in this course
 - Constant variables: `const double PI = 3.14159;`
 - Enumerations: `enum State { START, END };`
 - ◇ For integers only, useful for representing discrete states (Assignment 1!)
- Handling bad user input
 - Check return values of functions (e.g., `scanf`, `fopen`)
 - Report any errors to `stderr` using `fprintf` and exit gracefully

Overview

- 1 The build process
- 2 Running a C program
- 3 Code style
- 4 Summary

Food for thought

- What are some advantages and disadvantages of using a compiled language like C compared to an interpreted language like Python?
- How might the build process differ for a large project with multiple source files and dependencies?
 - Next precept!

Summary

- What happens under-the-hood when you write `gcc217 input.c`
- How to run a C program
- How to write well-structured C code
- Dates to remember:
 - Assignment 0: due Tue, 2/3 at 9pm (tonight!)
 - Assignment 1: due Tue, 2/10 at 9pm